

SMART CAMPUS & PLACEMENT PLATFORM

V3–V4 Roadmap — Production Backend to Full Platform

1. Purpose

This roadmap records the planned progression after the relational domain is stable. It separates production engineering from frontend completion so the Java backend remains the primary learning objective.

2. Project V3 — Production backend

Milestone	Planned learning / implementation
Security foundation	Spring Security request pipeline, authentication, authorization
Passwords	Secure password hashing and credential handling
JWT	Token authentication, claims, expiration and refresh strategy
Roles	Student, recruiter/company and admin authorization
Testing	JUnit, Mockito, MockMvc, integration tests
Logging	Useful application logs and diagnosis
Configuration	Profiles and externalized configuration
API documentation	OpenAPI/Swagger-style API documentation
Health/operations	Health and operational endpoints
Docker	Dockerfile, images, containers, Docker Compose
Deployment	Practical cloud deployment of backend + database

3. Security progression

- Understand authentication vs authorization.
- Understand Spring Security's filter-based request processing.
- Implement secure password storage.
- Implement login and authenticated requests.
- Introduce JWT issuance and validation.
- Protect endpoints according to roles.
- Keep security secrets externalized and out of Git.

4. Testing progression

- Unit-test important service/business behavior.
- Use Mockito where dependency isolation is useful.
- Use MockMvc for controller/API behavior.
- Add integration tests for important persistence/application flows.
- Prioritize tests that protect meaningful behavior instead of chasing an arbitrary coverage number.

5. Production configuration

- Use profiles for different environments.
- Externalize database credentials and application secrets.
- Keep example configuration safe for GitHub.

- Never commit passwords, JWT secrets, API keys, or tokens.

6. Docker and deployment

- Understand images vs containers.
- Create a Dockerfile for the backend.
- Run the application and PostgreSQL with Docker Compose during local development where useful.
- Pass environment-specific configuration through environment variables.
- Deploy the backend and database using a practical cloud setup.

7. Project V4 — Full Smart Campus & Placement Platform

React will be added after the backend API is stable enough to consume. The frontend is intentionally secondary to the Java backend objective.

Area	Planned functionality
Student dashboard	Profile, skills, projects, readiness, recommended jobs, applications
Company dashboard	Company profile, job creation, applicants, interview pipeline
Admin dashboard	Platform management and useful operational summaries
Authentication UI	Registration, login, protected routes, role-aware navigation
API integration	JSON communication with Spring Boot REST API
Placement intelligence	Readiness score, skill/job matching, recommendations

8. Placement intelligence — later optimization

- Begin with transparent rule-based readiness scoring.
- Compare student skills against job requirements.
- Recommend jobs using understandable matching rules.
- Measure usefulness before considering more advanced algorithms.

9. Final engineering polish

- Architecture diagram showing request-to-database flow.
- Database relationship diagram.
- Documented API endpoints with example requests/responses.
- Frontend screenshots or walkthrough.
- Docker setup and deployment instructions.
- README explaining important architectural decisions.
- Clean Git history showing meaningful evolution.
- Resume-ready project description focused on engineering outcomes.

10. Complete project evolution

Foundation → CRUD → validation/errors → DTO architecture → domain relationships → queries/pagination → transactions → security/JWT → automated testing → production configuration → Docker → deployment → React → placement intelligence → final documentation.

11. What should make the project stand out

- It demonstrates evolution rather than presenting one final code dump.

- The same codebase is repeatedly improved as new engineering problems appear.
- The developer can explain a request from HTTP through Spring MVC, services, repositories, JPA/Hibernate, JDBC and PostgreSQL, then back to the client.
- Architecture decisions are tied to concrete problems: coupling, validation, relationship management, transaction safety, security, testing, deployment and maintainability.
- The business domain is realistic enough to discuss beyond generic CRUD.

12. Documentation strategy going forward

Do not overwrite the historical snapshots. Keep each meaningful documentation update. A later document should record what changed from the previous checkpoint, what was learned, what was tested, what was improved or optimized, and what remains.

Final goal: an interview-explainable system with a working Java backend, relational data model, secure authentication/authorization, automated tests, containerization, deployment, React frontend, documented APIs, and a clear explanation of why the architecture evolved.